

MÓDULO 1 • PIPELINE DE DATOS Y PREPARACIÓN PROFESIONAL

Selección de Características y Tratamiento del Desbalanceo

Actividades prácticas — Feature Selection, métricas y SMOTE

Propósito

Aplicar técnicas de selección de características y estrategias para trabajar con datasets desbalanceados, prestando especial atención a la evaluación y a la prevención de Data Leakage.

Actividad 1. Identificar características relevantes

Imagina un modelo que debe predecir si un cliente abandonará un servicio. Analiza las siguientes variables:

edad → ¿Relevante / posiblemente relevante / irrelevante?

ingresos → ¿Relevante / posiblemente relevante / irrelevante?

color_favorito → ¿Relevante / posiblemente relevante / irrelevante?

cantidad_meses_cliente → ¿Relevante / posiblemente relevante / irrelevante?

cantidad_llamadas_soporte → ¿Relevante / posiblemente relevante / irrelevante?

tipo_plan → ¿Relevante / posiblemente relevante / irrelevante?

fecha_ultimo_contacto → ¿Relevante / posiblemente relevante / irrelevante?

ID_cliente → ¿Relevante / posiblemente relevante / irrelevante?

nombre_cliente → ¿Relevante / posiblemente relevante / irrelevante?

Justifica especialmente tus decisiones sobre ID_cliente y nombre_cliente.

Actividad 2. Detectar características redundantes

Supón que un dataset contiene:

```
ingresos_anuales
ingresos_mensuales
edad
cantidad_productos
```

- ¿Qué variables podrían contener información redundante?
- ¿Qué análisis realizarías para comprobarlo?
- ¿Eliminarías automáticamente alguna de ellas?
- ¿Qué información adicional necesitarías para decidir?

Actividad 3. Analizar correlaciones

Carga un dataset numérico y genera una matriz de correlación.

```
correlacion = df.corr(numeric_only=True)
print(correlacion)
```

Luego utiliza un mapa de calor:

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(correlacion, annot=True)
plt.show()
```

Identifica al menos dos pares de variables con alta relación y analiza si existe una posible redundancia.

Actividad 4. Aplicar VarianceThreshold

Crea un dataset que contenga una variable constante:

```
import pandas as pd

X = pd.DataFrame({
    'edad': [20, 25, 30, 35, 40],
    'ingresos': [20000, 25000, 30000, 35000, 40000],
    'constante': [1, 1, 1, 1, 1]
})
```

Aplica:

```
from sklearn.feature_selection import VarianceThreshold

selector = VarianceThreshold(threshold=0.0)
X_new = selector.fit_transform(X)
```

¿Qué característica fue eliminada? ¿Por qué?

Actividad 5. Aplicar SelectKBest

Utiliza un dataset de clasificación y selecciona las cinco mejores características.

```
from sklearn.feature_selection import SelectKBest, f_classif

selector = SelectKBest(score_func=f_classif, k=5)
X_new = selector.fit_transform(X, y)
```

- ¿Cuántas características tenía el dataset originalmente?
- ¿Cuántas quedaron después de la selección?
- ¿Cuáles fueron seleccionadas?
- ¿Qué puntuación obtuvo cada una?

Actividad 6. Mutual Information

Calcula la información mutua entre las características y la variable objetivo.

```
from sklearn.feature_selection import mutual_info_classif

scores = mutual_info_classif(
    X,
    y,
    random_state=42
)
```

Ordena las características desde la mayor hasta la menor puntuación y analiza el resultado.

Actividad 7. Recursive Feature Elimination

Utiliza RFE para seleccionar cinco características:

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
selector = RFE(model, n_features_to_select=5)
selector.fit(X, y)
```

- ¿Qué características fueron seleccionadas?
- ¿Qué características fueron descartadas?
- ¿Coinciden con las seleccionadas mediante SelectKBest?
- ¿Por qué pueden producir resultados diferentes?

Actividad 8. Importancia de características con Random Forest

Entrena un Random Forest y analiza la importancia de sus características.

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(random_state=42)
model.fit(X, y)

importancias = model.feature_importances_
```

Ordena las características según su importancia y selecciona las cinco primeras.

Actividad 9. Comparar métodos de selección

Compara los resultados obtenidos mediante:

- Correlación.
- VarianceThreshold.
- SelectKBest.
- Mutual Information.
- RFE.
- Random Forest.

Construye una tabla donde indiques qué características seleccionó cada método.

Actividad 10. Detectar desbalanceo

Considera la siguiente distribución:

```
Clase 0: 9500
Clase 1: 500
```

- Calcula el porcentaje de cada clase.
- ¿Existe desbalanceo?
- ¿Cuál es la clase minoritaria?
- ¿Qué problemas podría generar durante el entrenamiento?

Actividad 11. Analizar un dataset desbalanceado

Carga un dataset de clasificación y analiza la cantidad de observaciones de cada clase.

```
print(y.value_counts())
print(y.value_counts(normalize=True))
```

Representa gráficamente la distribución y escribe una conclusión.

Actividad 12. Analizar Accuracy, Precision, Recall y F1

Entrena un modelo de clasificación y calcula:

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

print('Accuracy:', accuracy_score(y_test, y_pred))
print('Precision:', precision_score(y_test, y_pred))
print('Recall:', recall_score(y_test, y_pred))
print('F1:', f1_score(y_test, y_pred))
```

Explica qué métrica consideras más importante para detectar correctamente la clase minoritaria y por qué.

Actividad 13. Interpretar una matriz de confusión

Supón que obtienes:

	Pred. 0	Pred. 1
Real 0	850	100
Real 1	30	20

- Identifica TP, TN, FP y FN.
- ¿Cuántos casos positivos reales fueron detectados?
- ¿Cuántos positivos fueron omitidos?
- ¿Qué problema observas?

Actividad 14. Class Weight

Entrena un modelo con y sin:

```
class_weight='balanced'
```

Compara Precision, Recall y F1-score.

Actividad 15. Random Oversampling

Utiliza RandomOverSampler para aumentar la representación de la clase minoritaria.

```
from imblearn.over_sampling import RandomOverSampler

ros = RandomOverSampler(random_state=42)
X_resampled, y_resampled = ros.fit_resample(X_train, y_train)
```

Compara la cantidad de observaciones antes y después.

Actividad 16. Random Undersampling

Utiliza RandomUnderSampler:

```
from imblearn.under_sampling import RandomUnderSampler

rus = RandomUnderSampler(random_state=42)
X_resampled, y_resampled = rus.fit_resample(X_train, y_train)
```

Analiza qué ventaja y qué posible desventaja presenta esta estrategia.

Actividad 17. Aplicar SMOTE

Aplica SMOTE exclusivamente sobre el conjunto de entrenamiento.

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(
    X_train,
    y_train
)
```

Comprueba la distribución de las clases antes y después.

Actividad 18. Comparar estrategias de balanceo

Entrena el mismo modelo utilizando:

- Sin balanceo.
- Class Weight.
- Random Oversampling.
- Random Undersampling.
- SMOTE.

Construye una tabla con Accuracy, Precision, Recall y F1-score.

Actividad 19. Detectar Data Leakage

Determina si las siguientes situaciones son correctas o incorrectas:

Situación	¿Correcto?	Justificación
Aplicar SMOTE antes de train_test_split.		
Aplicar SMOTE solamente a X_train e y_train.		
Seleccionar características usando todo el dataset antes de separar.		

Situación	¿Correcto?	Justificación
Utilizar Pipeline para integrar la selección.		

Actividad 20. Construir un Pipeline con SMOTE

Construye un Pipeline que incluya escalado, SMOTE y un modelo de clasificación.

```
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('model', LogisticRegression(max_iter=1000))
])
```

Entrena el pipeline y evalúa el resultado sobre X_test.

Actividad 21. Combinar Feature Selection y SMOTE

Construye un flujo que realice:

- Selección de características.
- Escalado.
- SMOTE.
- Modelo de clasificación.

Utiliza un Pipeline y explica por qué el orden elegido es adecuado.

Actividad 22. Comparar modelos y estrategias

Construye una tabla final donde compares al menos tres configuraciones:

Configuración	Accuracy	Precision	Recall	F1
Modelo sin balanceo				
Modelo + class_weight				
Modelo + SMOTE				

Escribe una conclusión justificando qué configuración elegirías y por qué.

Actividad Integradora — Preparación profesional de un dataset

Consigna

Selecciona un dataset de clasificación que presente varias características y, preferentemente, cierto grado de desbalanceo entre sus clases. Realiza un proceso completo de preparación profesional de los datos.

El trabajo deberá incluir:

- 1. Descripción del problema.
- 2. Descripción del dataset.
- 3. Distribución de la variable objetivo.
- 4. Identificación de características relevantes e irrelevantes.
- 5. Análisis de características redundantes.
- 6. Aplicación de al menos un método Filter.
- 7. Aplicación de al menos un método Wrapper o Embedded.
- 8. Comparación de los métodos de selección.
- 9. Separación correcta entre entrenamiento y prueba.
- 10. Selección de una estrategia para el desbalanceo.
- 11. Aplicación de Class Weight, Oversampling, Undersampling o SMOTE.
- 12. Comparación de las estrategias utilizadas.
- 13. Evaluación mediante Precision, Recall y F1-score.
- 14. Análisis de la matriz de confusión.
- 15. Construcción de un Pipeline.
- 16. Verificación de posibles casos de Data Leakage.
- 17. Conclusiones y justificación de las decisiones.

Autoevaluación

1. ¿Qué es Feature Selection?

Respuesta: Es el proceso de seleccionar las características más útiles para un modelo.

2. ¿Cuál es la diferencia entre Feature Engineering y Feature Selection?

Respuesta: Feature Engineering crea o transforma variables; Feature Selection elige cuáles conservar.

3. ¿Qué hace VarianceThreshold?

Respuesta: Elimina características con una variabilidad inferior al umbral establecido.

4. ¿Para qué sirve SelectKBest?

Respuesta: Selecciona las K características con mejor puntuación según una función estadística.

5. ¿Qué significa que un dataset esté desbalanceado?

Respuesta: Que las clases tienen cantidades significativamente diferentes de observaciones.

6. ¿Por qué Accuracy puede ser engañosa?

Respuesta: Porque un modelo puede favorecer la clase mayoritaria y obtener alta Accuracy sin detectar correctamente la minoritaria.

7. ¿Qué mide Recall?

Respuesta: La proporción de casos positivos reales que el modelo consigue detectar.

8. ¿Qué es SMOTE?

Respuesta: Una técnica de sobremuestreo que genera ejemplos sintéticos de la clase minoritaria.

9. ¿Dónde debe aplicarse SMOTE?

Respuesta: Sobre el conjunto de entrenamiento, dentro del proceso de entrenamiento.

10. ¿Qué problema evita un Pipeline?

Respuesta: Ayuda a mantener las transformaciones dentro del flujo correcto y reduce riesgos de Data Leakage.

Cierre del Módulo 1

Del dato al modelo.

A lo largo del Módulo 1 se construyó un pipeline profesional de preparación de datos: comprensión del problema, EDA, limpieza, imputación, Feature Engineering, selección de características y tratamiento del desbalanceo.

El siguiente paso será utilizar estos datos preparados para construir y optimizar modelos supervisados.